

Отчет по заданию 4 (сдано 18.02)

Цель задания

Целью задания являлась разработка программного прототипа Telegram-бота для автоматизированного приёма заявок клиентов на разработку Telegram-ботов компанией Vox-Bot.

Отчет по проделанной работе:

В рамках задания требовалось реализовать Telegram-бота, обеспечивающего:

1. интерактивное меню для пользователей;
2. выбор типа бота (индивидуальный или шаблонный);
3. сбор контактных данных и описания задачи;
4. сохранение заявок в базе данных;
5. возможность просмотра и редактирования заявок;
6. уведомление администратора о новых заявках.

В процессе разработки были использованы следующие технологии:

1. язык программирования Python 3;
2. библиотека python-telegram-bot;
3. SQLite для хранения данных;
4. Telegram Bot API;

Алгоритм работы бота:

1. Пользователь запускает бота командой /start.
2. Бот отображает главное меню с помощью клавиатуры Telegram.
3. Пользователь выбирает тип бота и отправляет описание задачи.
4. Бот сохраняет заявку в базе данных SQLite.
5. Администратор получает уведомление о новой заявке.
6. Пользователь может редактировать ранее созданную заявку.

Код программы:

```
from telegram import (
    Update, ReplyKeyboardMarkup, InlineKeyboardMarkup, InlineKeyboardButton
)
from telegram.ext import (
    Application, CommandHandler, MessageHandler, filters,
    ContextTypes, ConversationHandler, CallbackQueryHandler
)
import uuid
import sqlite3
import os

TOKEN = os.getenv -
ADMINS = -
SUPPORT_LINK = -
DB_FILE = "requests.db"
```

```

CHOOSING, CHOOSING_TYPE, TEMPLATE_CHOICE, REQUEST, EDIT = range(5)

main_menu = ReplyKeyboardMarkup(
    [
        ["✉ Оставить заявку", "ℹ Узнать подробнее"],
        ["🛒 Мой заказ", "NEW Новый бот"],
        ["☎ Связаться с поддержкой"]
    ],
    resize_keyboard=True
)

type_menu = ReplyKeyboardMarkup(
    [
        ["☐ Индивидуальный бот", "📄 Шаблонный бот"],
        ["← Главное меню"]
    ],
    resize_keyboard=True
)

template_menu = ReplyKeyboardMarkup(
    [
        ["📄 Заявка-бот"],
        ["📄 Визитка-бот"],
        ["📄 Бронирование"],
        ["💬 Поддержка"],
        ["📢 Рассылка"],
        ["← Назад"]
    ],
    resize_keyboard=True
)

after_request_menu = ReplyKeyboardMarkup([["⬅ Внести изменения", "← Главное меню"]], resize_keyboard=True)
my_orders_menu = ReplyKeyboardMarkup([["⬅ Внести изменения"], ["← Главное меню"]], resize_keyboard=True)

# ----- БАЗА ДАННЫХ -----
def init_db():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS requests (
            id TEXT PRIMARY KEY,
            user_id INTEGER,
            type TEXT,
            description TEXT,
            contact TEXT
        )
    """)
    conn.commit()
    conn.close()

def save_request_to_db(user_id: int, bot_type: str, description: str,
contact: str) -> str:
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    request_id = str(uuid.uuid4())[:8]
    cursor.execute("INSERT INTO requests (id, user_id, type, description,
contact) VALUES (?, ?, ?, ?, ?)",
        (request_id, user_id, bot_type, description, contact))
    conn.commit()
    conn.close()

```

```

        return request_id

def update_request_in_db(request_id: str, new_description: str, contact: str)
-> None:
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute("UPDATE requests SET description = ?, contact = ? WHERE id
= ?",
                    (new_description, contact, request_id))
    conn.commit()
    conn.close()

def get_requests_by_user(user_id: int):
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute("SELECT id, user_id, type, description, contact FROM
requests WHERE user_id = ? ORDER BY rowid", (user_id,))
    rows = cursor.fetchall()
    conn.close()
    return rows

def get_all_requests():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute("SELECT id, user_id, type, description, contact FROM
requests ORDER BY rowid")
    rows = cursor.fetchall()
    conn.close()
    return rows

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "Привет! 🖐 Я бот для приёма заявок на создание Telegram-
ботов.\nВыберите действие:",
        reply_markup=main_menu
    )
    return CHOOSING

async def choice(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    user_id = update.message.from_user.id

    if text == "✉ Оставить заявку":
        await update.message.reply_text("Выберите тип заявки:",
reply_markup=type_menu)
        return CHOOSING_TYPE

    elif text == "ℹ Узнать подробнее":
        await update.message.reply_text(
            "Мы создаём Telegram-ботов для бизнеса: визитки, бронирование,
поддержка, рассылки.\n"
            "По вопросам: https://t.me/nasvaozo",
            reply_markup=main_menu
        )
        return CHOOSING

    elif text == "🛒 Мой заказ":
        requests = get_requests_by_user(user_id)
        if not requests:
            await update.message.reply_text("У вас пока нет заявок.",
reply_markup=main_menu)
            return CHOOSING

```

```

        msg = "Ваши заявки:\n\n"
        for r in requests:
            msg += f"ID: {r[0]}\nТип: {r[2]}\nОписание: {r[3]}\nКонтакт: {r[4]}\n{'-'*20}\n"
            await update.message.reply_text(msg, reply_markup=my_orders_menu)
        return EDIT

    elif text == "🆕 Новый бот":
        return await start(update, context)

    elif text == "☎ Связаться с поддержкой":
        await update.message.reply_text(
            f"Связаться с поддержкой: {SUPPORT_LINK}",
            reply_markup=main_menu
        )
        return CHOOSING

    else:
        await update.message.reply_text("Пожалуйста, выберите одну из кнопок.", reply_markup=main_menu)
        return CHOOSING

async def choose_type(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    if text == "← Главное меню":
        return await start(update, context)

    if text == "👤 Индивидуальный бот":
        context.user_data["type"] = "Индивидуальный бот"
        await update.message.reply_text(
            "Опишите задачу и оставьте контакт.",
            reply_markup=after_request_menu
        )
        return REQUEST

    if text == "📄 Шаблонный бот":
        await update.message.reply_text("Выберите один из доступных шаблонов:", reply_markup=template_menu)
        return TEMPLATE_CHOICE

    await update.message.reply_text("Пожалуйста, выберите один из вариантов.", reply_markup=type_menu)
    return CHOOSING_TYPE

templates = {
    "📄 Заявка-бот": {"desc": "Сбор контактов клиента.", "demo": "https://t.me/zaivkademobot"},
    "📄 Визитка-бот": {"desc": "Информация о компании.", "demo": "https://t.me/vizitka_demobot"},
    "📅 Бронирование": {"desc": "Запись на услуги.", "demo": "https://t.me/demobronbot"},
    "☎ Поддержка": {"desc": "Чат с оператором.", "demo": "https://t.me/suportdemo_bot"},
    "📢 Рассылка": {"desc": "Массовые уведомления.", "demo": "https://t.me/rassilkademobot"},
}

async def template_choice(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    if text == "← Назад":

```

```

        await update.message.reply_text("Выберите тип заявки:",
reply_markup=type_menu)
        return CHOOSING_TYPE

    if text in templates:
        context.user_data["type"] = text
        keyboard = [
            [InlineKeyboardButton("👁️ Посмотреть демо",
url=templates[text]["demo"])],
            [InlineKeyboardButton("✉️ Оставить заявку на этот шаблон",
callback data="leave request")]]
        ]
        await update.message.reply_text(
            f"{text}\nОписание: {templates[text]['desc']}",
            reply_markup=InlineKeyboardMarkup(keyboard)
        )
        return TEMPLATE_CHOICE

    await update.message.reply_text("Пожалуйста, выберите один из шаблонов.",
reply_markup=template_menu)
    return TEMPLATE_CHOICE

async def template_request_callback(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer()
    bot_type = context.user_data.get("type", "Не выбран")
    await query.message.reply_text(
        f"Вы выбрали: {bot_type}. Опишите ваши пожелания или оставьте
контакт.",
        reply_markup=after_request_menu
    )
    return REQUEST

async def request(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    if text == "← Главное меню":
        return await start(update, context)
    if text == "✎ Внести изменения":
        # Если пользователь по ошибке нажал эту кнопку в режиме REQUEST —
        # просто повторим просьбу
        await update.message.reply_text("Напишите текст заявки или вернитесь
в меню.", reply_markup=after_request_menu)
        return REQUEST

    user = update.message.from_user
    bot_type = context.user_data.get("type", "Не выбран")
    contact = f"@{user.username}" if user.username else f"ID: {user.id}"

    # Если редактируем существующую заявку
    edit_id = context.user_data.get("edit_id")
    if edit_id:
        update_request_in_db(edit_id, text, contact)
        for admin_id in ADMINS:
            await context.bot.send_message(
                admin_id,
                f"✎ Обновлённая заявка (ID: {edit_id}) от {user.first_name}
({user.username}): \n"
                f"Тип: {bot_type}\nНовое описание: {text}"
            )
        await update.message.reply_text(f"Заявка обновлена ✅\nID: {edit_id}",
reply_markup=main_menu)
        context.user_data.pop("edit_id", None)

```

```

        return CHOOSING

    # Создаём новую заявку
    request_id = save_request_to_db(user.id, bot_type, text, contact)
    for admin_id in ADMINS:
        await context.bot.send_message(
            admin_id,
            f"📩 Новая заявка (ID: {request_id}) от {user.first_name} (@{user.username} - {user.id}): \n"
            f"Тип: {bot_type} \n Описание: {text}"
        )
    await update.message.reply_text(
        f"Спасибо! Ваша заявка принята. \n Ваш ID: {request_id} \n Вы можете её изменить:",
        reply_markup=after_request_menu
    )
    # При желании сразу предлагаем редактирование последней
    context.user_data["edit_id"] = request_id
    return EDIT

async def edit_request(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = update.message.text
    user_id = update.message.from_user.id

    if text == "← Главное меню":
        return await start(update, context)

    if text == "✎ Внести изменения":
        requests = get_requests_by_user(user_id)
        if not requests:
            await update.message.reply_text("У вас нет заявок для изменения.", reply_markup=main_menu)
            return CHOOSING
        # Берём последнюю заявку пользователя
        last_request_id = requests[-1][0]
        context.user_data["edit_id"] = last_request_id
        await update.message.reply_text(
            f"Изменение заявки ID: {last_request_id} \n Отправьте новый текст:",
            reply_markup=ReplyKeyboardMarkup([["← Главное меню"]],
            resize_keyboard=True)
        )
        return REQUEST

    # Любой другой текст в этом состоянии трактуем как новый текст заявки
    context.user_data["edit_id"] = context.user_data.get("edit_id") # не теряем id
    return await request(update, context)

async def cancel(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Диалог завершён. Для нового запроса используйте /start.", reply_markup=main_menu)
    return ConversationHandler.END

# ----- АДМИН-КОМАНДА: /all_requests -----
async def all_requests(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_id = update.effective_user.id
    if user_id not in ADMINS:
        await update.message.reply_text("🚫 У вас нет доступа к этой команде.")
        return

    rows = get_all_requests()
```

```

if not rows:
    await update.message.reply_text("📄 Заявок пока нет.")
    return

# Формируем текст и отправляем батчами, чтобы не упереться в лимит
телеграма
chunk = []
total_len = 0
for r in rows:
    line = (
        f"ID: {r[0]}\n"
        f"Пользователь: {r[1]}\n"
        f"Тип: {r[2]}\n"
        f"Описание: {r[3]}\n"
        f"Контакт: {r[4]}\n"
        f"{'-'*30}\n"
    )
    if total_len + len(line) > 3500:
        await update.message.reply_text("".join(chunk))
        chunk, total_len = [], 0
    chunk.append(line)
    total_len += len(line)
if chunk:
    await update.message.reply_text("".join(chunk))

def main():
    init_db()
    app = Application.builder().token(TOKEN).build()

    conv_handler = ConversationHandler(
        entry_points=[CommandHandler("start", start)],
        states={
            CHOOSING: [
                MessageHandler(filters.TEXT & ~filters.COMMAND, choice),
                CommandHandler("all_requests", all_requests) # <- доступно в
этом состоянии
            ],
            CHOOSING_TYPE: [
                MessageHandler(filters.TEXT & ~filters.COMMAND, choose_type),
                CommandHandler("all_requests", all_requests)
            ],
            TEMPLATE_CHOICE: [
                MessageHandler(filters.TEXT & ~filters.COMMAND,
template choice),
                CallbackQueryHandler(template_request_callback,
pattern="^leave_request$"),
                CommandHandler("all_requests", all_requests)
            ],
            REQUEST: [
                MessageHandler(filters.TEXT & ~filters.COMMAND, request),
                CommandHandler("all_requests", all_requests)
            ],
            EDIT: [
                MessageHandler(filters.TEXT & ~filters.COMMAND,
edit_request),
                CommandHandler("all_requests", all_requests)
            ],
        },
        fallbacks=[
            CommandHandler("cancel", cancel),
            CommandHandler("start", start),
            CommandHandler("all_requests", all_requests) # <- и как fallback
на всякий

```

```
        ],
    )

    app.add_handler(conv_handler)

    # Дополнительно можно добавить и глобально (не обязательно, но не
    # мешает)
    app.add_handler(CommandHandler("all_requests", all_requests))

    print("Бот запущен...")
    app.run_polling()

if __name__ == "__main__":
    main()
```

Вывод

Разработка Telegram-бота позволила освоить практические навыки программирования на Python, работы с Telegram Bot API и базами данных SQLite. Созданное программное решение демонстрирует возможности автоматизации бизнес-процессов и повышения эффективности взаимодействия с клиентами.